

```
# =====
# ned.hal -- LinuxCNC HAL for the "ned" mill
# Phase 1: reproduce everything the existing Fagor 8055 machine does.
#
# Machine: X is a GANTRY (two servos: X "gantry servo 1" + W "gantry servo 2",
#          tandem) ; Y cross ; Z vertical ; one VFD spindle.
#          Full drawbar tool change + toolsetter + sensors.
# Hardware: Mesa 7I97T (Ethernet, hm2_eth) + 7I85S (DB25) + 7I84U (sserial).
# Kinematics (set in ned.ini): trivkins coordinates=XYZX -> JOINTS=4
#          joint 0 = X (gantry side 1)   joint 1 = Y
#          joint 2 = Z                   joint 3 = X (gantry side 2 / "W")
#
# Wiring source of truth: docs/wiring_to_hal_guide.md, docs/fagor_8055_axes.md.
# HAL/man reference parked offline in docs/linuxcnc/.
#
# DESIGN NOTE (read this):
# All machine LOGIC below is written against named SIGNALS and is independent
# of the Mesa hardware. The ONLY card-dependent part is the block titled
# "PHYSICAL I/O BINDINGS" near the bottom. After the firmware bitfile is
# loaded, confirm every hm2_7i97.0.* pin name and index with:
#     halcmd show pin hm2_7i97.0
# The board may enumerate as "hm2_7i97" or "hm2_7i97t" -- match what show pin
# reports and adjust the prefix throughout if needed.
#
# On the 7I97 the +/-10V analog outputs are PWMGEN instances routed through the
# card's PWM->analog stage (there is no "analogout" module). pwmgen.NN.scale is
# the value that maps to full +/-10 V; pwmgen.NN.value is the command.
# =====

# -----
# 1. LOAD REALTIME COMPONENTS
# -----

# --- Mesa driver -----
# board_ip and the config= firmware string MUST match your card + bitfile.
# num_encoders/num_pwmgens are clamped to what the firmware provides; confirm
# with show pin. sserial_port_0 enables the 7I84U on the smart-serial port
# (auto-detected). Tune this line once the real bitfile is flashed.
loadrt hostmot2
loadrt hm2_eth board_ip="10.10.10.10" config="num_encoders=4 num_pwmgens=5 sserial_port_0=00xxxxxx"

setp hm2_7i97.0.watchdog.timeout_ns 5000000

# --- Motion controller -----
loadrt [KINS]KINEMATICS
loadrt [EMCMOT]EMCMOT servo_period_nsec=[EMCMOT]SERVO_PERIOD num_joints=[KINS]JOINTS num_spindles=[TRAJ]SPINDLES

# --- Per-joint position PID (analog velocity-mode servos) -----
loadrt pid names=pid.x,pid.y,pid.z,pid.w

# --- Logic gates for the spindle + drawbar interlock -----
loadrt and2 names=spindle.permit,spindle.cw,spindle.ccw

# -----
# 2. REALTIME THREADS (servo thread only -- no base thread in Phase 1)
# -----
addf hm2_7i97.0.read          servo-thread
addf motion-command-handler  servo-thread
addf motion-controller        servo-thread
addf pid.x.do-pid-calcs       servo-thread
addf pid.y.do-pid-calcs       servo-thread
addf pid.z.do-pid-calcs       servo-thread
addf pid.w.do-pid-calcs       servo-thread
addf spindle.permit           servo-thread
addf spindle.cw               servo-thread
addf spindle.ccw              servo-thread
addf hm2_7i97.0.write         servo-thread

# =====
# 3. AXIS / JOINT LOGIC (signal-level; hardware bound in section 7)
# =====
# Each joint: motion <-> PID <-> (encoder feedback in, pwmgen volts out).
# PID output is in VOLTS; pwmgen scale (section 7) maps 10 -> full +/-10 V.

# --- Joint 0 : X (gantry side 1) -----
```

```

ncu:net      Lindex0c HAL[ /1511; X-gantry/1/2 1 Spindle 1:cutchange)
setp pid.x.Pgain      [JOINT_0]P
setp pid.x.Igain      [JOINT_0]I
setp pid.x.Dgain      [JOINT_0]D
setp pid.x.bias       [JOINT_0]BIAS
setp pid.x.FF0        [JOINT_0]FF0
setp pid.x.FF1        [JOINT_0]FF1
setp pid.x.FF2        [JOINT_0]FF2
setp pid.x.deadband   [JOINT_0]DEADBAND
setp pid.x.maxoutput  [JOINT_0]MAX_OUTPUT
setp pid.x.error-previous-target true

net x-enable    joint.0.amp-enable-out => pid.x.enable      sig-x-amp-enable
net x-pos-cmd   joint.0.motor-pos-cmd  => pid.x.command
net x-pos-fb    pid.x.feedback          <= sig-x-enc-pos
net x-pos-fb    => joint.0.motor-pos-fb
net x-output    pid.x.output            => sig-x-vel-volts

# --- Joint 1 : Y (cross) -----
setp pid.y.Pgain      [JOINT_1]P
setp pid.y.Igain      [JOINT_1]I
setp pid.y.Dgain      [JOINT_1]D
setp pid.y.bias       [JOINT_1]BIAS
setp pid.y.FF0        [JOINT_1]FF0
setp pid.y.FF1        [JOINT_1]FF1
setp pid.y.FF2        [JOINT_1]FF2
setp pid.y.deadband   [JOINT_1]DEADBAND
setp pid.y.maxoutput  [JOINT_1]MAX_OUTPUT
setp pid.y.error-previous-target true

net y-enable    joint.1.amp-enable-out => pid.y.enable      sig-y-amp-enable
net y-pos-cmd   joint.1.motor-pos-cmd  => pid.y.command
net y-pos-fb    pid.y.feedback          <= sig-y-enc-pos
net y-pos-fb    => joint.1.motor-pos-fb
net y-output    pid.y.output            => sig-y-vel-volts

# --- Joint 2 : Z (vertical) -----
setp pid.z.Pgain      [JOINT_2]P
setp pid.z.Igain      [JOINT_2]I
setp pid.z.Dgain      [JOINT_2]D
setp pid.z.bias       [JOINT_2]BIAS
setp pid.z.FF0        [JOINT_2]FF0
setp pid.z.FF1        [JOINT_2]FF1
setp pid.z.FF2        [JOINT_2]FF2
setp pid.z.deadband   [JOINT_2]DEADBAND
setp pid.z.maxoutput  [JOINT_2]MAX_OUTPUT
setp pid.z.error-previous-target true

net z-enable    joint.2.amp-enable-out => pid.z.enable      sig-z-amp-enable
net z-pos-cmd   joint.2.motor-pos-cmd  => pid.z.command
net z-pos-fb    pid.z.feedback          <= sig-z-enc-pos
net z-pos-fb    => joint.2.motor-pos-fb
net z-output    pid.z.output            => sig-z-vel-volts

# --- Joint 3 : X (gantry side 2 / "W") -----
# Same X axis as joint 0 via trivkins XYZX. Homes to its own switch so the
# gantry can square itself (see ned.ini HOME_SEQUENCE for joints 0 & 3).
setp pid.w.Pgain      [JOINT_3]P
setp pid.w.Igain      [JOINT_3]I
setp pid.w.Dgain      [JOINT_3]D
setp pid.w.bias       [JOINT_3]BIAS
setp pid.w.FF0        [JOINT_3]FF0
setp pid.w.FF1        [JOINT_3]FF1
setp pid.w.FF2        [JOINT_3]FF2
setp pid.w.deadband   [JOINT_3]DEADBAND
setp pid.w.maxoutput  [JOINT_3]MAX_OUTPUT
setp pid.w.error-previous-target true

net w-enable    joint.3.amp-enable-out => pid.w.enable      sig-w-amp-enable
net w-pos-cmd   joint.3.motor-pos-cmd  => pid.w.command
net w-pos-fb    pid.w.feedback          <= sig-w-enc-pos
net w-pos-fb    => joint.3.motor-pos-fb
net w-output    pid.w.output            => sig-w-vel-volts

# =====
# 4. SPINDLE (VFD: +/-10V magnitude on AI2, FWD/REV via S1/S2)
# =====
# VFD takes a 0..10V magnitude (speed-out-abs) plus discrete FWD/REV.

```

```
# Interlock: spindle is permitted only while the drawbar is UP (tool clamped).

net spindle-on          motion.spindle.0.on          => spindle.permit.in0
net drawbar-clamped     sig-drawbar-up              => spindle.permit.in1
net spindle-permitted   spindle.permit.out          => sig-spindle-vel-enable

net spindle-permitted => spindle.cw.in0
net spindle-fwd       motion.spindle.0.forward     => spindle.cw.in1
net spin-cw           spindle.cw.out               => sig-spin-cw

net spindle-permitted => spindle.ccw.in0
net spindle-rev       motion.spindle.0.reverse     => spindle.ccw.in1
net spin-ccw          spindle.ccw.out               => sig-spin-ccw

net spindle-speed      motion.spindle.0.speed-out-abs => sig-spindle-vel-volts

# Spindle feedback / faults -- monitored (available to GUI + interlocks).
# spindle-running comes from the VFD "running" contact via R1.
net spindle-running    sig-spindle-running         => motion.spindle.0.is_oriented
# NOTE: is_oriented is only a convenient sink so the signal is live in HAL;
#       re-point it to a real consumer (GUI LED / at-speed logic) as desired.

# =====
# 5. ESTOP, ENABLE, LIMITS, HOMING
# =====
# E-stop is a HARDWARE daisy chain; HAL only reads the result (e-stop-released
# HIGH when the whole chain is intact) and asserts drive-enable to R5.
net estop-ext          sig-estop-released          => iocontrol.0.emc-enable-in
net drive-enable       iocontrol.0.user-enable-out => sig-drive-enable

# Limit switches are NC / fail-safe: contact OPENS at the limit (or on a broken
# wire), so "at limit" = input LOW. The PHYSICAL block (section 7) feeds these
# signals from the INVERTED (-not) input pins so an open reads as "at limit".
net lim-x-neg          sig-limit-x-back            => joint.0.neg-lim-sw-in joint.3.neg-lim-sw-in
net lim-x-pos          sig-limit-x-front           => joint.0.pos-lim-sw-in joint.3.pos-lim-sw-in
net lim-y-neg          sig-limit-y-left            => joint.1.neg-lim-sw-in
net lim-y-pos          sig-limit-y-right           => joint.1.pos-lim-sw-in
net lim-z-neg          sig-limit-z-bottom          => joint.2.neg-lim-sw-in
net lim-z-pos          sig-limit-z-top             => joint.2.pos-lim-sw-in

# No separate home switches were traced -> home off the limit switches.
# (ned.ini sets HOME_USE_INDEX / HOME offsets; gantry joints 0 & 3 home
# independently so the bridge can square.)
net lim-x-neg          => joint.0.home-sw-in joint.3.home-sw-in
net lim-y-neg          => joint.1.home-sw-in
net lim-z-pos          => joint.2.home-sw-in

# Monitored safety inputs (faithful to the Fagor: these did not drop machine
# e-stop, they were status the CNC read). Exposed here for the GUI / optional
# interlocks; wire into emc-enable-in via and2 later if you want them to E-stop.
#   sig-air-pressure-ok   HIGH = air OK
#   sig-vfd-fault        HIGH = VFD faulted
#   sig-spindle-overtemp  HIGH = spindle thermostat tripped

# =====
# 6. TOOL CHANGE (drawbar + taper purge + toolsetter + probe)
# =====
# The physical sequence (purge -> release -> wait drawbar-down -> swap ->
# clamp -> wait drawbar-up) is best driven by a Python M6 REMAP declared in
# ned.ini ([RS274NGC] REMAP=M6 ...). The remap drives the motion.digital-out-*
# pins below and reads the motion.digital-in-* sensors. HAL here just exposes
# the live I/O and acks the iocontrol handshake so the config runs standalone.

# iocontrol handshake (immediate ack; the remap supersedes the motion of it).
net tool-prep-loop     iocontrol.0.tool-prepare => iocontrol.0.tool-prepared
net tool-change-loop   iocontrol.0.tool-change => iocontrol.0.tool-changed
setp iocontrol.0.tool-prep-number 0

# Tool-change actuators -> exposed as digital outputs (M64/M65 P# or remap):
#   digital-out-00 = drawbar RELEASE (ODRAW)      *65
#   digital-out-01 = drawbar CLAMP (OCLAMP)       *63
#   digital-out-02 = spindle taper PURGE         *64
#   digital-out-03 = toolsetter DEPLOY           *61
#   digital-out-04 = chip blow-off (air blast)    *55
net drawbar-release    motion.digital-out-00 => sig-drawbar-release
net drawbar-clamp      motion.digital-out-01 => sig-drawbar-clamp
```

```
net taper-purge      motion.digital-out-02 => sig-taper-purge
net toolsetter-deploy motion.digital-out-03 => sig-toolsetter-deploy
net chip-blowoff     motion.digital-out-04 => sig-chip-blowoff
```

```
# Tool-change sensors -> exposed as digital inputs (M66 or remap):
```

```
# digital-in-00 = drawbar UP (clamped) *69
# digital-in-01 = drawbar DOWN (released)*70
# digital-in-02 = rack position *68
net drawbar-up      sig-drawbar-up    => motion.digital-in-00
net drawbar-down    sig-drawbar-down  => motion.digital-in-01
net rack-position   sig-rack-pos      => motion.digital-in-02
```

```
# Tool-length probe (toolsetter contact, via R10) -> motion probe for G38.x.
```

```
net probe-in        sig-tool-probe   => motion.probe-input
```

```
# =====
```

```
# 7. PHYSICAL I/O BINDINGS <-- THE ONLY CARD-DEPENDENT SECTION
```

```
# =====
```

```
# Confirm EVERY name/index here against 'halcmd show pin hm2_7i97.0'.
```

```
# Chosen distribution (a wiring-design choice, revise freely):
```

```
# - Axis encoders + analog (pwmgen) : 7I97T native
# - All field digital I/O           : 7I84U via smart serial (.7i84.0.0.*)
```

```
# --- Encoders (machine units via [JOINT_n]INPUT_SCALE) -----
```

```
# Standard encoder setup lines (match pncconf / hm2-servo reference).
```

```
setp hm2_7i97.0.encoder.00.counter-mode 0
setp hm2_7i97.0.encoder.00.filter 1
setp hm2_7i97.0.encoder.00.index-invert 0
setp hm2_7i97.0.encoder.00.index-mask 0
setp hm2_7i97.0.encoder.00.index-mask-invert 0
setp hm2_7i97.0.encoder.00.scale [JOINT_0]INPUT_SCALE
setp hm2_7i97.0.encoder.01.counter-mode 0
setp hm2_7i97.0.encoder.01.filter 1
setp hm2_7i97.0.encoder.01.scale [JOINT_1]INPUT_SCALE
setp hm2_7i97.0.encoder.02.counter-mode 0
setp hm2_7i97.0.encoder.02.filter 1
setp hm2_7i97.0.encoder.02.scale [JOINT_2]INPUT_SCALE
setp hm2_7i97.0.encoder.03.counter-mode 0
setp hm2_7i97.0.encoder.03.filter 1
setp hm2_7i97.0.encoder.03.scale [JOINT_3]INPUT_SCALE
net sig-x-enc-pos <= hm2_7i97.0.encoder.00.position
net sig-y-enc-pos <= hm2_7i97.0.encoder.01.position
net sig-z-enc-pos <= hm2_7i97.0.encoder.02.position
net sig-w-enc-pos <= hm2_7i97.0.encoder.03.position
```

```
# Optional: home-to-index. Uncomment per joint that has an encoder index (Z marker)
```

```
# and set HOME_USE_INDEX = YES in ned.ini for that joint.
```

```
#net x-index-enable joint.0.index-enable <=> hm2_7i97.0.encoder.00.index-enable
```

```
#net y-index-enable joint.1.index-enable <=> hm2_7i97.0.encoder.01.index-enable
```

```
#net z-index-enable joint.2.index-enable <=> hm2_7i97.0.encoder.02.index-enable
```

```
#net w-index-enable joint.3.index-enable <=> hm2_7i97.0.encoder.03.index-enable
```

```
# --- Analog velocity commands (pwmgen -> +/-10 V) -----
```

```
# output-type 1 = PWM on pin1, Dir on pin2 (matches hm2-servo reference).
```

```
# pwmgen.NN.scale = [JOINT_n]OUTPUT_SCALE sets the value that maps to full
```

```
# +/-10 V; tune in ned.ini. Confirm bipolar swing on your bitfile; enable
```

```
# offset-mode if the firmware needs it for a true bipolar output.
```

```
setp hm2_7i97.0.pwmgen.00.output-type 1
setp hm2_7i97.0.pwmgen.01.output-type 1
setp hm2_7i97.0.pwmgen.02.output-type 1
setp hm2_7i97.0.pwmgen.03.output-type 1
setp hm2_7i97.0.pwmgen.00.scale [JOINT_0]OUTPUT_SCALE
setp hm2_7i97.0.pwmgen.01.scale [JOINT_1]OUTPUT_SCALE
setp hm2_7i97.0.pwmgen.02.scale [JOINT_2]OUTPUT_SCALE
setp hm2_7i97.0.pwmgen.03.scale [JOINT_3]OUTPUT_SCALE
setp hm2_7i97.0.pwmgen.pwm_frequency 40000
```

```
net sig-x-vel-volts    => hm2_7i97.0.pwmgen.00.value
net sig-y-vel-volts    => hm2_7i97.0.pwmgen.01.value
net sig-z-vel-volts    => hm2_7i97.0.pwmgen.02.value
net sig-w-vel-volts    => hm2_7i97.0.pwmgen.03.value
net sig-x-amp-enable   => hm2_7i97.0.pwmgen.00.enable
net sig-y-amp-enable   => hm2_7i97.0.pwmgen.01.enable
net sig-z-amp-enable   => hm2_7i97.0.pwmgen.02.enable
net sig-w-amp-enable   => hm2_7i97.0.pwmgen.03.enable
```

```
# --- Spindle analog (pwmgen 04, scale = max RPM -> 10 V) -----
setp hm2_7i97.0.pwmgen.04.scale [SPINDLE_0]MAX_FORWARD_VELOCITY
net sig-spindle-vel-volts => hm2_7i97.0.pwmgen.04.value
net sig-spindle-vel-enable => hm2_7i97.0.pwmgen.04.enable

# --- Digital INPUTS (7I84U sserial; NC limits & friends use the -not pin) ---
net sig-estop-released    <= hm2_7i97.0.7i84.0.0.input-00    # *6  e-stop chain OK (HIGH=released)
net sig-limit-x-back      <= hm2_7i97.0.7i84.0.0.input-01-not  # *24 gantry-X back  (NC)
net sig-limit-x-front     <= hm2_7i97.0.7i84.0.0.input-02-not  # *25 gantry-X front (NC)
net sig-limit-y-left      <= hm2_7i97.0.7i84.0.0.input-03-not  # *26 Y left  (NC)
net sig-limit-y-right     <= hm2_7i97.0.7i84.0.0.input-04-not  # *27 Y right (NC)
net sig-limit-z-top       <= hm2_7i97.0.7i84.0.0.input-05-not  # *28 Z top   (NC)
net sig-limit-z-bottom    <= hm2_7i97.0.7i84.0.0.input-06-not  # *29 Z bottom (NC)
net sig-spindle-running   <= hm2_7i97.0.7i84.0.0.input-07      # *36 VFD running (via R1)
net sig-air-pressure-ok   <= hm2_7i97.0.7i84.0.0.input-08      # *37 air pressure (HIGH=OK)
net sig-vfd-fault        <= hm2_7i97.0.7i84.0.0.input-09      # *38 VFD fault
net sig-spindle-overtemp  <= hm2_7i97.0.7i84.0.0.input-10      # *39 spindle thermostat
net sig-tool-probe        <= hm2_7i97.0.7i84.0.0.input-11      # *54 toolsetter contact (via R10)
net sig-rack-pos          <= hm2_7i97.0.7i84.0.0.input-12      # *68 rack position
net sig-drawbar-up        <= hm2_7i97.0.7i84.0.0.input-13      # *69 drawbar UP (clamped)
net sig-drawbar-down      <= hm2_7i97.0.7i84.0.0.input-14      # *70 drawbar DOWN (released)

# --- Digital OUTPUTS (7I84U sserial) -----
net sig-drive-enable      => hm2_7i97.0.7i84.0.0.output-00    # *8  drive-enable -> R5 coil
net sig-spin-cw           => hm2_7i97.0.7i84.0.0.output-01    # *40 SPIN-CW -> R6
net sig-spin-ccw          => hm2_7i97.0.7i84.0.0.output-02    # *41 SPIN-CCW -> R7
net sig-chip-blowoff      => hm2_7i97.0.7i84.0.0.output-03    # *55 chip blow-off -> R9
net sig-toolsetter-deploy => hm2_7i97.0.7i84.0.0.output-04    # *61 toolsetter deploy
net sig-drawbar-clamp     => hm2_7i97.0.7i84.0.0.output-05    # *63 drawbar clamp (OCLAMP)
net sig-taper-purge       => hm2_7i97.0.7i84.0.0.output-06    # *64 taper air purge (OBLOWOFF)
net sig-drawbar-release   => hm2_7i97.0.7i84.0.0.output-07    # *65 drawbar release (ODRAW)

# =====
# END ned.hal
# To launch you also need ned.ini (KINS, EMCOT, TRAJ, JOINT_0..3, SPINDLE_0,
# AXIS_X/Y/Z, RS274NGC remap, etc.). The HAL above references these INI vars:
# [KINS]KINEMATICS [KINS]JOINTS
# [EMCOT]EMCOT [EMCOT]SERVO_PERIOD
# [TRAJ]SPINDLES
# [JOINT_0..3] P I D BIAS FF0 FF1 FF2 DEADBAND MAX_OUTPUT INPUT_SCALE OUTPUT_SCALE
# [SPINDLE_0]MAX_FORWARD_VELOCITY
# =====
```